

PWM IP Core

Software API Reference

Revision 1.0 | Duval MAMBOU | ENIB — Bretagne INP

Target processor	BSP	Language
Nios II (Altera)	HAL (Altera)	C
Files	Dependencies	
pwm_regs.h, pwmAPI.h/c	io.h, sys/alt_irq.h	

Contents

1 Overview	2
2 File: pwm_regs.h — Register Access Layer	2
2.1 Register Address Offsets	2
2.2 I/O Access Macros	2
2.3 Bit Masks and Offsets	2
3 File: pwmAPI.h / pwmAPI.c — High-Level API	3
3.1 Instance Macros	3
3.1.1 PWM_ISR_INSTALL	3
3.1.2 PWM_INSTANCE_INIT	3
3.2 API Functions	3
3.2.1 PWM_Init	3
3.2.2 PWM_Start	4
3.2.3 PWM_Stop	4
3.2.4 PWM_State	4
3.2.5 PWM_PulseState	4
3.2.6 PWM_WritePeriod	4
3.2.7 PWM_ReadPeriod	5
3.2.8 PWM_WriteDutyCycle	5
3.2.9 PWM_ReadDutyCycle	5
3.2.10 PWM_IrqEnable	5
3.2.11 PWM_IrqDisable	6
3.2.12 PWM_IsIrqPending	6
3.2.13 PWM_WriteNbCycles	6
3.2.14 PWM_ReadNbCycles	6
4 Usage Example	6
5 API Function Summary	7
6 Revision History	7

Overview

The PWM software driver provides a high-level C API for controlling the PWM IP Core peripheral from a Nios II processor running on the Altera HAL (Hardware Abstraction Layer). It abstracts all direct register accesses and exposes human-readable functions operating in milliseconds and percentages rather than raw clock cycle counts.

The driver is organized in three layers:

1. **pwm_regs.h** — Low-level register access macros and bit field definitions
2. **pwmAPI.h** / **pwmAPI.c** — High-level API functions and instance macros
3. **main_testAPI.c** — Application-level test program

File: pwm_regs.h — Register Access Layer

Register Address Offsets

Macro	Value	Access	Description
STATUS	0	RO	PWM state, pulse state, IRQ request
SETCTRL	0	WO	Set Start / IrqEn bits
PERIOD	1	R/W	PWM period (clock cycles)
DCYCLE	2	R/W	Duty cycle (clock cycles)
NBCYCLES	3	R/W	Number of cycles before IRQ
CLRCTRL	4	WO	Clear Start / IrqEn bits

I/O Access Macros

Macro	Description
IORD_PWM_STATUS(base)	Read Status register
IOWR_PWM_SETCTRL(base, data)	Write SetCtrl register
IORD_PWM_PERIOD(base)	Read Period register
IOWR_PWM_PERIOD(base, data)	Write Period register
IORD_PWM_DCYCLE(base)	Read DCycle register
IOWR_PWM_DCYCLE(base, data)	Write DCycle register
IORD_PWM_NBCYCLES(base)	Read NbCycles register
IOWR_PWM_NBCYCLES(base, data)	Write NbCycles register
IOWR_PWM_CLRCTRL(base, data)	Write ClrCtrl register

Bit Masks and Offsets

Macro	Value	Bit	Description
PWM_STATUS_PWMSTATE_MSK	0x00000001	0	PWM running state
PWM_STATUS_PWMSTATE_OFST	0		
PWM_STATUS_PULSESTATE_MSK	0x00000002	1	PWM output level
PWM_STATUS_PULSESTATE_OFST	1		
PWM_STATUS_IRQREQ_MSK	0x00000004	2	IRQ request flag
PWM_STATUS_IRQREQ_OFST	2		
PWM_CTRL_START_MSK	0x00000001	0	Start/Stop control
PWM_CTRL_IRQEN_MSK	0x00000002	1	IRQ enable control
PWM_NBCYCLES_MSK	0x0000FFFF	—	Valid bits mask

File: pwmAPI.h / pwmAPI.c — High-Level API

Instance Macros

PWM_ISR_INSTALL

Syntax: PWM_ISR_INSTALL(name, cb)

Description: Defines the Interrupt Service Routine for a PWM instance. This macro generates a function named `name_ISR` that reads the Status register (clearing the IrqRq flag) and calls the user callback `cb`. Must be placed at file scope, outside any function.

Parameter	Description
<code>name</code>	Instance name as defined in Qsys (uppercase, e.g. USERPWM)
<code>cb</code>	User callback function of type <code>void cb(void)</code>

```
// Example
void my_callback(void) {
    printf("IRQ received\n");
}
PWM_ISR_INSTALL(USERPWM, my_callback);
```

PWM_INSTANCE_INIT

Syntax: PWM_INSTANCE_INIT(name, period, dutycycle)

Description: Initializes a PWM instance by calling `PWM_Init` with all parameters automatically resolved from the Qsys-generated `system.h` symbols (`name_BASE`, `name_FREQ`, `name_IRQ`, `name_IRQ_INTERRUPT_CONTROLLER_ID`). Also registers the ISR previously defined by `PWM_ISR_INSTALL`.

Parameter	Description
<code>name</code>	Instance name as defined in Qsys (uppercase)
<code>period</code>	PWM period in milliseconds
<code>dutycycle</code>	Initial duty cycle in percent (0–100)

Return: 0 on success, -1 on error.

```
// Initialize USERPWM with 100ms period, 20% duty cycle
PWM_INSTANCE_INIT(USERPWM, 100, 20);
```

API Functions

PWM_Init

Prototype:

```
char PWM_Init(unsigned long BaseAddress, unsigned long PeriodValue,
              unsigned char DutyCycleValue, unsigned long
              ClockFrequency,
              long PWM_ic_id, long PWM_irq, alt_isr_func isr);
```

Description: Initializes the PWM peripheral. Stops the modulator if running, converts period (ms) and duty cycle (%) to clock cycle counts, writes the parameters to the hardware

registers, verifies the write, and registers the ISR.

Parameter	Description
BaseAddress	Peripheral base address from <code>system.h</code>
PeriodValue	Desired PWM period in ms
DutyCycleValue	Initial duty cycle in % (0–100)
ClockFrequency	Clock frequency in Hz driving the PWM
PWM_ic_id	Interrupt controller ID
PWM_irq	IRQ number (-1 to skip ISR registration)
isr	Pointer to interrupt service routine

Return: 0 on success, -1 on error.

Conversion formulas:

$$\text{Period}_{reg} = \frac{f_{Clk}}{1000} \times \text{PeriodValue}_{ms} \quad \text{DCycle}_{reg} = \frac{\text{Period}_{reg}}{100} \times \text{DutyCycleValue}_{\%}$$

PWM_Start

Prototype: `char PWM_Start(unsigned long BaseAddress);`

Description: Starts the PWM modulator by setting the `Start` bit via `SetCtrl`. Verifies the operation by reading back the `PWMState` bit from the Status register.

Return: 0 on success, -1 if the PWM could not be started.

PWM_Stop

Prototype: `char PWM_Stop(unsigned long BaseAddress);`

Description: Stops the PWM modulator by clearing the `Start` bit via `ClrCtrl`. Verifies the operation by reading back the `PWMState` bit.

Return: 0 on success, -1 if the PWM could not be stopped.

PWM_State

Prototype: `char PWM_State(unsigned long BaseAddress);`

Description: Returns the current running state of the PWM modulator.

Return: 1 if running, 0 if stopped.

PWM_PulseState

Prototype: `char PWM_PulseState(unsigned long BaseAddress);`

Description: Returns the current logic level of the PWM output signal (`OutPWM`).

Return: 1 if output is high, 0 if output is low.

PWM_WritePeriod

Prototype:

```
char PWM_WritePeriod(unsigned long BaseAddress,
                    unsigned long ClockFrequency,
                    unsigned long PeriodValue);
```

Description: Updates the PWM period while preserving the current duty cycle ratio. If the modulator is running, it is stopped before the update and restarted afterward. The new duty cycle register value is recalculated to maintain the same percentage ratio.

Return: 0 on success, -1 on error.

PWM_ReadPeriod**Prototype:**

```
unsigned long PWM_ReadPeriod(unsigned long BaseAddress,
                             unsigned long ClockFrequency);
```

Description: Reads the Period register and converts its value to milliseconds.

Return: Period in ms (rounded to nearest integer).

Conversion: $T_{ms} = \text{Period}_{reg} \times 1000f_{Clk} + 0.5$

PWM_WriteDutyCycle**Prototype:**

```
char PWM_WriteDutyCycle(unsigned long BaseAddress,
                        unsigned char DutyCycleValue);
```

Description: Updates the duty cycle. The value is expressed as a percentage (0–100). The function reads the current Period register value to compute the corresponding clock cycle count.

Return: 0 on success, -1 if DutyCycleValue > 100 or write verification fails.

PWM_ReadDutyCycle

Prototype: unsigned long PWM_ReadDutyCycle(unsigned long BaseAddress);

Description: Reads the DCycle register and converts it to a percentage relative to the current Period.

Return: Duty cycle in % (0–100, rounded).

Conversion: $\alpha\% = 100 \times \text{DCycle}_{reg} \text{Period}_{reg} + 0.5$

PWM_IrqEnable

Prototype: void PWM_IrqEnable(unsigned long BaseAddress);

Description: Enables interrupt generation by setting the `IrqEn` bit via `SetCtrl`. Interrupts will be triggered each time `CntCyc` reaches `NbCyc`.

PWM_IrqDisable

Prototype: `void PWM_IrqDisable(unsigned long BaseAddress);`

Description: Disables interrupt generation by clearing the `IrqEn` bit via `ClrCtrl`.

PWM_IsIrqPending

Prototype: `char PWM_IsIrqPending(unsigned long BaseAddress);`

Description: Reads the `IrqRq` bit from the Status register.

Return: 1 if an interrupt is pending, 0 otherwise.

Warning: Reading the Status register clears the `IrqRq` flag in hardware. Calling this function when `IrqRq` = 1 consumes the pending interrupt.

PWM_WriteNbCycles

Prototype:

```
char PWM_WriteNbCycles(unsigned long BaseAddress,
                      unsigned char NbCycles);
```

Description: Sets the number of complete PWM periods that must elapse before asserting an interrupt request. Only the lower 16 bits are significant.

Return: 0 on success, -1 if write verification fails.

PWM_ReadNbCycles

Prototype: `unsigned long PWM_ReadNbCycles(unsigned long BaseAddress);`

Description: Reads the `NbCycles` register and masks the result to 16 bits.

Return: Number of cycles configured for interrupt generation.

Usage Example

The following example is extracted from `main_testAPI.c`. It demonstrates the complete initialization, control, and interrupt workflow for a PWM instance named `USERPWM`.

```
#include "DEO_CV_includes.h"
#include "pwmAPI.h"

static int irq_count = 0;

// User callback      called on each PWM interrupt
void my_callback(void) {
    printf("IRQ #%d\n", irq_count++);
}
```

```
// Install ISR for USERPWM instance (file scope)
PWM_ISR_INSTALL(USERPWM, my_callback);

int main(void) {
    // Initialize: 100ms period, 20% duty cycle
    PWM_INSTANCE_INIT(USERPWM, 100, 20);

    // Start the PWM
    PWM_Start(USERPWM_BASE);

    // Change duty cycle to 50%
    PWM_WriteDutyCycle(USERPWM_BASE, 50);

    // Configure interrupt every 100 cycles, then enable
    PWM_WriteNbCycles(USERPWM_BASE, 100);
    PWM_IrqEnable(USERPWM_BASE);

    // Wait for 10 interrupts
    while (irq_count < 10) { ; }

    // Disable interrupts and stop
    PWM_IrqDisable(USERPWM_BASE);
    PWM_Stop(USERPWM_BASE);
    return 0;
}
```

API Function Summary

Function	Return	Description
PWM_Init(...)	char	Full initialization with ISR registration
PWM_Start(base)	char	Start the PWM modulator
PWM_Stop(base)	char	Stop the PWM modulator
PWM_State(base)	char	Read running state (0/1)
PWM_PulseState(base)	char	Read output signal level (0/1)
PWM_WritePeriod(...)	char	Set period in ms
PWM_ReadPeriod(...)	ulong	Get period in ms
PWM_WriteDutyCycle(...)	char	Set duty cycle in %
PWM_ReadDutyCycle(base)	ulong	Get duty cycle in %
PWM_IrqEnable(base)	void	Enable interrupt generation
PWM_IrqDisable(base)	void	Disable interrupt generation
PWM_IsIrqPending(base)	char	Check and clear IRQ flag
PWM_WriteNbCycles(...)	char	Set cycles count before IRQ
PWM_ReadNbCycles(base)	ulong	Get cycles count before IRQ

Revision History

Rev.	Date	Author	Description
1.0	April 2026	Duval MAMBOU	Initial release